

---

---

# **Estrutura de Dados:**

# **Lista Dinâmica Encadeada**

Prof. Fábio Medeiros Faria

---

---

# Definição

- Tipo de lista onde cada elemento aponta para seu sucessor;
- Usa **ponteiro** especial para o primeiro elemento da lista e uma indicação de final de lista;
- Cada elemento é tratado como um ponteiro que é alocado dinamicamente, conforme os dados são inseridos;
- Para guardar o primeiro elemento , faz-se uso de um “ponteiro para ponteiro”;
- “ponteiro para ponteiro” guarda o endereço de um outro ponteiro

# Vantagens

- Melhor utilização dos recursos de memória;
- não precisam ser movimentados os dados nas operações inserções e remoções;

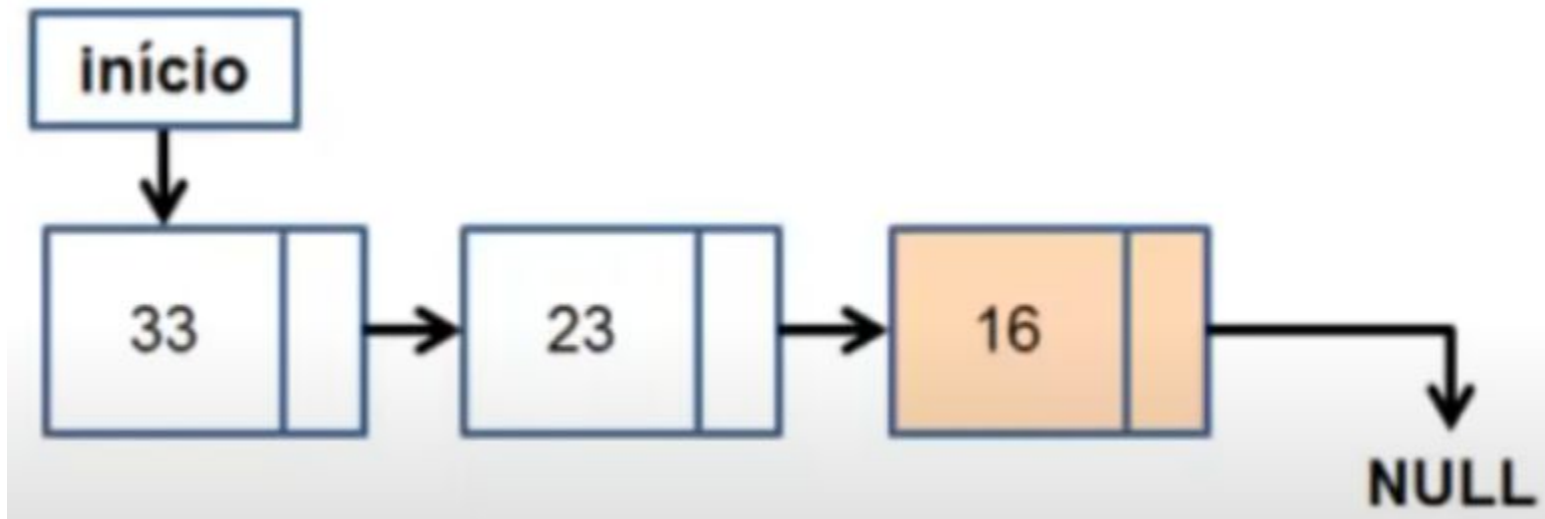
# Quando utilizar?

Quando não há necessidade de garantir um espaço mínimo para execução do aplicativo;

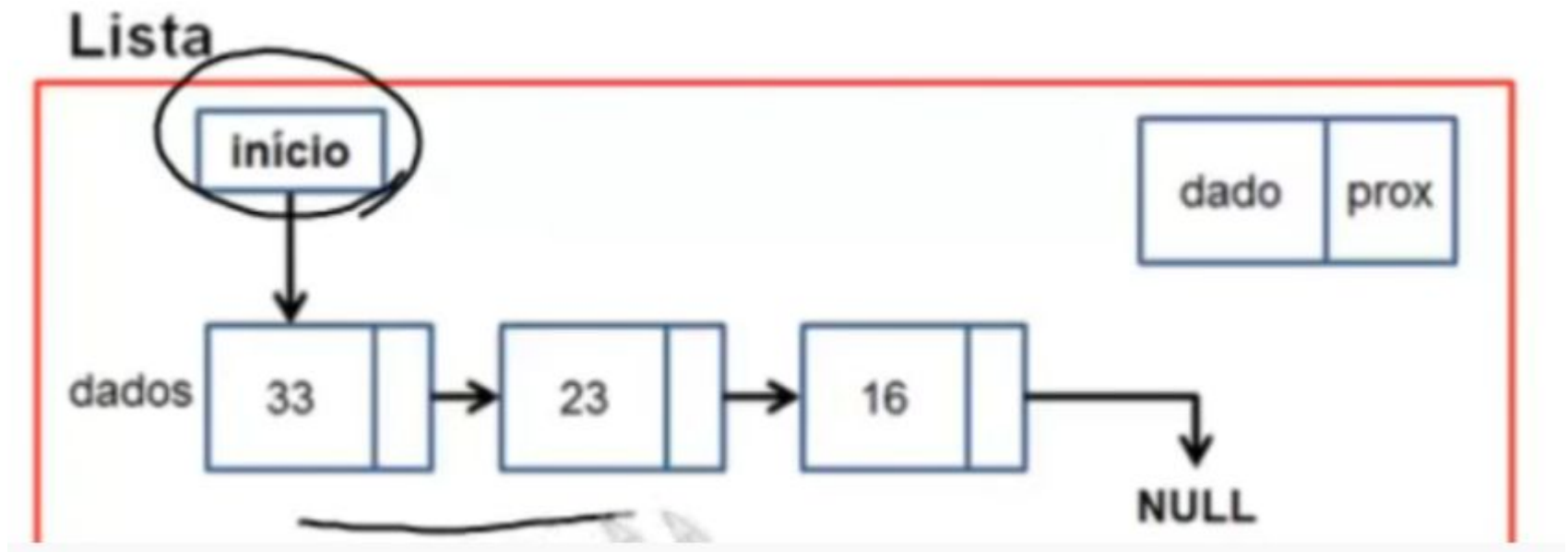
Inserção/remoção lista **ordenadas** são operações mais frequentes;

# Desvantagens

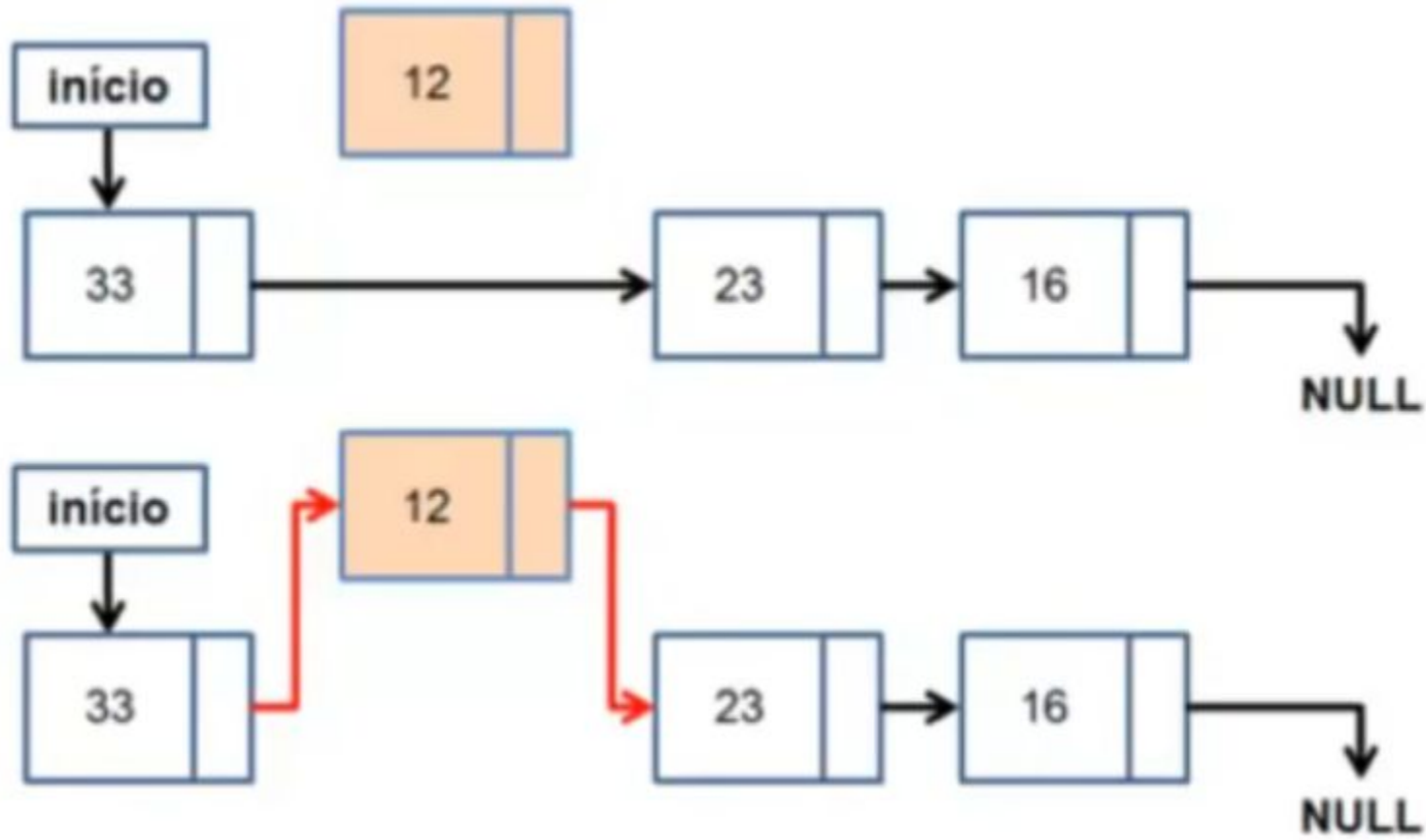
- Acesso indireto aos elementos;
- Necessidade de percorrer a lista para acessar um elemento



# Lista Dinâmica Encadeada



# Lista Dinâmica Encadeada



# Lista Dinâmica Encadeada: estrutura de dados

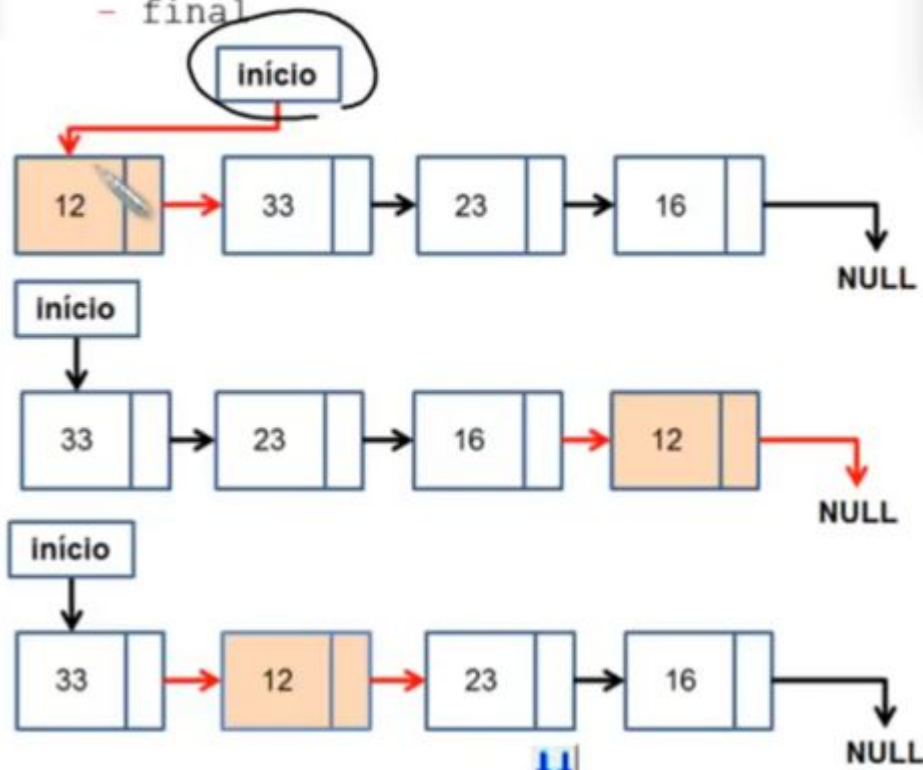
```
6  struct aluno {
7      int matricula;
8      char nome[30];
9      float nota1;
10     float nota2;
11     float nota3;
12 };
13
14 typedef struct aluno Aluno;
15
16 struct elemento {
17     struct aluno dados;
18     struct elemento *proximo;
19 };
20
21 typedef struct elemento* Lista;
22
23 typedef struct elemento Elemento;
```



# Lista Dinâmica Encadeada: inserção

Existem 3 tipos de inserção:

- início
- meio
- final

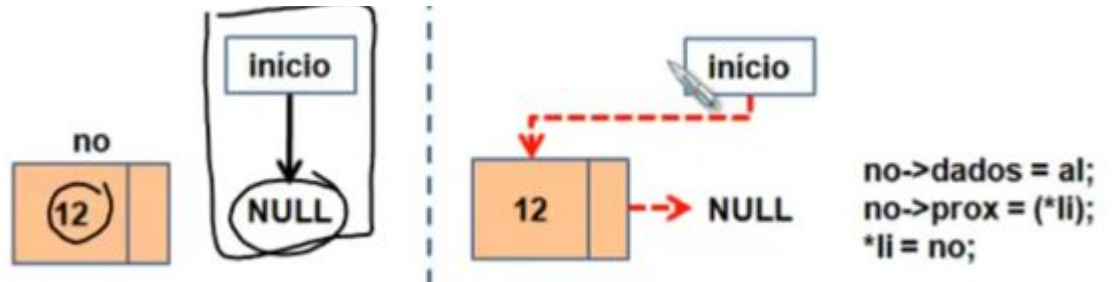


# Lista Dinâmica Encadeada: criar a lista

```
45  Lista* criarLista() {
46
47      Lista *lista = (Lista*) malloc(sizeof(Lista));
48
49      //conseguiu alocar memória
50      if(lista != NULL) {
51          *lista = NULL;
52      }
53
54      return lista;
55
56 }
```

# Lista Dinâmica Encadeada: inserção início

```
58 int inserirInicioLista(Lista *lista, Aluno novoAluno) {
59
60     //verica se a lista já existe
61     if(lista == NULL) {
62         return 0; //false
63     }
64
65     Elemento *novoElemento = (Elemento*) malloc(sizeof(Elemento));
66
67     if(novoElemento == NULL) {
68         return 0; //false pq não foi possível alocar novo elemento
69     }
70
71     novoElemento->dados = novoAluno;
72     novoElemento->proximo = (*lista);
73     *lista = novoElemento;
74
75     return 1; //true deu certo
76 }
```



# Lista Dinâmica Encadeada: imprimir lista

```
78 void imprimirLista(Lista *lista) {
79
80     if(lista == NULL) {
81         printf("Lista nulla :(");
82         return;
83     }
84
85     Elemento *elemento = *lista;
86
87     printf("\n##### LISTA DE ALUNOS #####");
88     while(elemento != NULL) {
89
90         printf("\nMatricula: %d", elemento->dados.matricula);
91         printf("\tNome: %s", elemento->dados.nome);
92         printf("\n-----");
93         printf("\nNota 1: %f", elemento->dados.nota1);
94         printf("\tNota 2: %f", elemento->dados.nota2);
95         printf("\tNota 3: %f", elemento->dados.nota3);
96         printf("\n-----\n");
97
98         elemento = elemento->proximo;
99
100     }
```

# Lista Dinâmica Encadeada: teste

```
29  int main(void) {
30
31      Aluno aluno1 = {111, "Joaozinho", 5.5, 7.7, 9.9};
32      Aluno aluno2 = {222, "Maria", 10.0, 5.0, 10.0};
33      Aluno aluno3 = {333, "Pedrinho", 9.0, 6.0, 3.0};
34
35      Lista *listaAlunos = criarLista();
36
37      inserirInicioLista(listaAlunos, aluno1);
38      inserirInicioLista(listaAlunos, aluno2);
39      inserirInicioLista(listaAlunos, aluno3);
40
41      imprimirLista(listaAlunos);
42
43 }
```

##### LISTA DE ALUNOS #####

Matricula: 111 Nome: Joaozinho

-----  
Nota 1: 5.500000 Nota 2: 7.700000 Nota 3: 9.900000  
-----

Matricula: 222 Nome: Maria

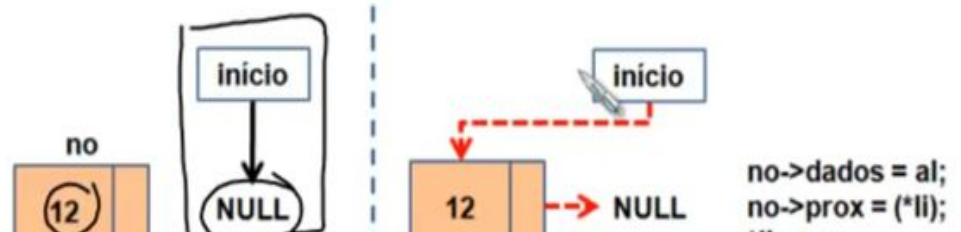
-----  
Nota 1: 10.000000 Nota 2: 5.000000 Nota 3: 10.000000  
-----

Matricula: 333 Nome: Pedrinho

-----  
Nota 1: 9.000000 Nota 2: 6.000000 Nota 3: 3.000000  
-----

# Lista Dinâmica Encadeada: inserção fim

```
int inserirFimLista(Lista* lista, Aluno novoAluno){
    if(lista == NULL)
        return 0;
    Elemento *elemento;
    elemento = (Elemento*) malloc(sizeof(Elemento));
    if(elemento == NULL)
        return 0;
    elemento->aluno = novoAluno;
    elemento->proximo = NULL;
    if((*lista) == NULL){//lista vazia: insere inicio
        *lista = elemento;
    }else{
        Elemento *aux;;
        aux = *lista;
        while(aux->proximo != NULL){
            aux = aux->proximo;
        }
        aux->proximo = elemento;
    }
    return 1;
}
```



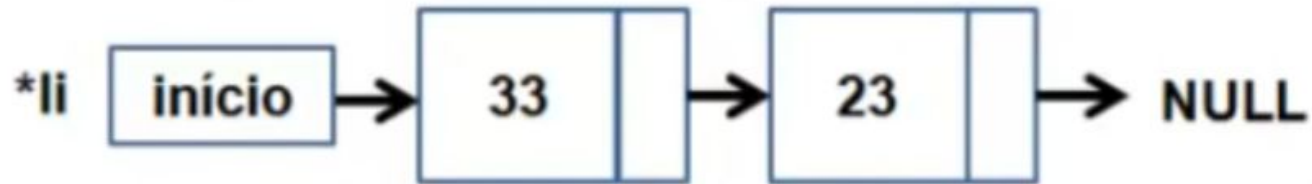


# Lista Dinâmica Encadeada: inserção fim

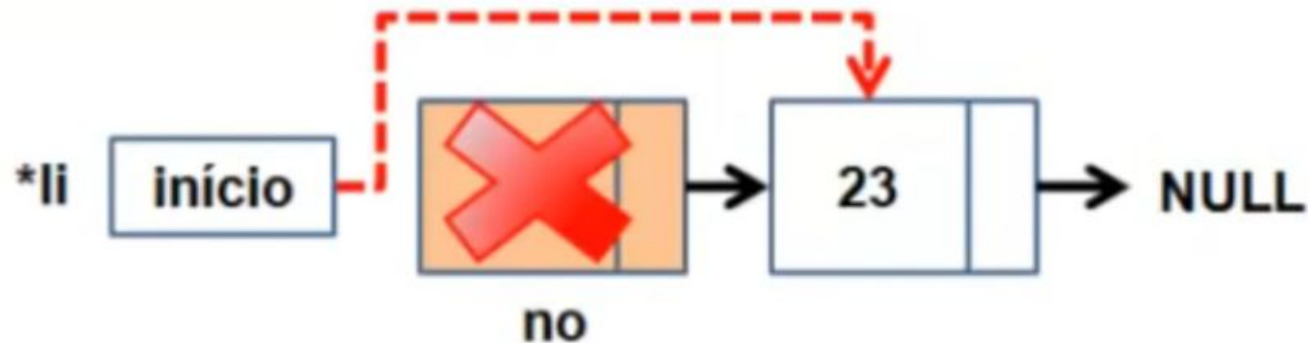
```
110 int inserirFim(Lista *lista, Aluno novoAluno) {
111     if(lista == NULL) {
112         printf("InserirFim: lista null");
113         return 0;
114     }
115     Elemento *elemento = (Elemento*) malloc(sizeof(Elemento));
116     if(elemento == NULL) {
117         printf("Inserir fim: elemento null");
118         return 0;
119     }
120     elemento->dados = novoAluno;
121     elemento->proximo = NULL; //inserção fim: último aponta null
122
123     //verifica se a lista eh vazia
124     if((*lista) == NULL) {
125         *lista = elemento;
126     } else {
127         //encontra último
128         Elemento *aux = *lista;
129         while(aux->proximo != NULL) {
130             aux = aux->proximo;
131         }
132         //ate entao ultimo elemento aponta para o novo ultimo
133         aux->proximo = elemento;
134     }
135     return 1;
136 }
```

# Lista Dinâmica Encadeada

```
void libera_lista(Lista* li);
```



```
no = *li;  
*li = (*li)->prox;  
free(no);
```



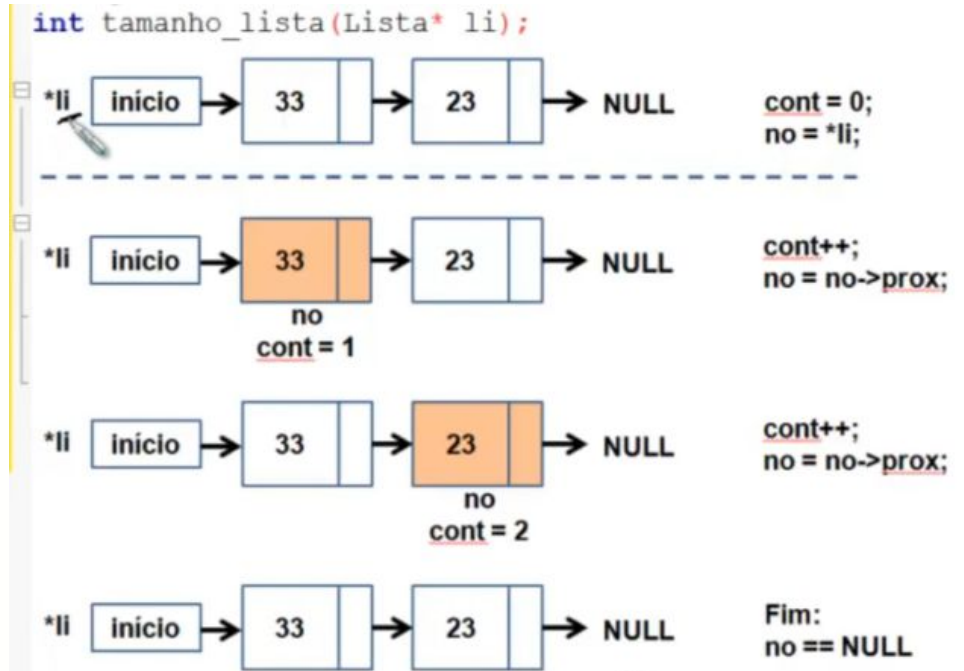


## Lista Dinâmica Encadeada: limpar a lista

```
74 void libera_lista(Lista* lista){
75     if(lista != NULL){
76         Elemento* elemento;
77         while((*lista) != NULL){
78             elemento = *lista;
79             *lista = (*lista)->proximo;
80             free(elemento);
81         }
82         free(lista);
83     } *lista = NULL;
84 }
```

# Lista Dinâmica Encadeada: tamanho lista

```
143 int tamanhoLista(Lista *lista) {
144
145     if(lista == NULL) {
146         return 0;
147     }
148
149     int cont = 0;
150
151     Elemento *elemento = *lista;
152
153     while(elemento != NULL) {
154         cont++; //acresce 1
155         elemento = elemento->proximo;
156     }
157
158     return cont;
159 }
```



# Lista Dinâmica Encadeada: lista vazia?

```
161  int ehListaVazia(Lista *lista) {
162
163      if(lista == NULL) {
164          return 1; //é vazia
165      }
166
167      if(*lista == NULL) {
168          return 1; //vazia
169      }
170
171      return 0; //não é vazia
172  }
173
```

# Lista Dinâmica Encadeada: remoção início

```
int removerInicioLista(Lista* lista){  
    if(lista == NULL)  
        return 0;  
    if((*lista) == NULL)//lista vazia  
        return 0;  
  
    Elemento *atual = *lista;  
    *lista = atual->proximo;  
    free(atual);  
    return 1;  
}
```

# Lista Dinâmica Encadeada: remoção fim

```
213 int removerFimLista(Lista *lista) {
214
215     if(ehListaVazia(lista)) {
216         return 0; //false: não pode remover
217     }
218
219     Elemento *ultimoElemento = *lista;
220
221     Elemento *elementoAnterior;
222
223     //encontrei último elemento
224     //penúltimo = anterior
225     while(ultimoElemento->proximo != NULL) {
226         elementoAnterior = ultimoElemento;
227         ultimoElemento = ultimoElemento->proximo;
228     }
229
230     if(ultimoElemento == (*lista)) {
231         *lista = ultimoElemento->proximo;
232     } else {
233         elementoAnterior->proximo = ultimoElemento->proximo;
234     }
235
236     free(ultimoElemento);
237
238     return 1;
239
240 }
```

# Lista Dinâmica Encadeada: remoção por propriedade

```
243 int removerListaMatricula(Lista* lista, int codigoMatricula){
244     if(lista == NULL)
245         return 0;
246     if((*lista) == NULL)//lista vazia
247         return 0;
248     Elemento *anterior, *atual = *lista;
249     while(atual != NULL && atual->dados.matricula != codigoMatricula){
250         anterior = atual;
251         atual = atual->proximo;
252     }
253     if(atual == NULL)//não encontrado
254         return 0;
255
256     if(atual == *lista)//remover o primeiro?
257         *lista = atual->proximo;
258     else
259         anterior->proximo = atual->proximo;
260     free(atual);
261     return 1;
262 }
```

# Lista Dinâmica Encadeada: inserção ordenada

```
127 int insere_lista_ordenada(Lista* lista, Aluno aluno){
128     if(lista == NULL)
129         return 0;
130     Elemento *elemento = (Elemento*) malloc(sizeof(elemento));
131     if(elemento == NULL)
132         return 0;
133     elemento->dados = aluno;
134     if((*lista) == NULL){//lista vazia: insere início
135         elemento->proximo = NULL;
136         *lista = elemento;
137         return 1;
138     }
139     else{
140         Elemento *anterior, *atual = *lista;
141         while(atual != NULL && atual->dados.matricula < aluno.matricula){
142             anterior = atual;
143             atual = atual->proximo;
144         }
145         if(atual == *lista){//insere início
146             elemento->proximo = (*lista);
147             *lista = elemento;
148         }else{
149             elemento->proximo = atual;
150             anterior->proximo = elemento;
151         }
152         return 1;
153     }
154 }
```

